System-One and Jev

If the term is unfamiliar to you, here is the short version: System 1 is the fast, intuitive, and emotional system, while System 2 is the slow, deliberate, and logical one. Daniel Kahneman, Nobel Prize laureate in Economics, coined this term in his psychology work on judgment and decision making, which challenged the idea that humans are rational decision makers. (Kahneman, *Thinking, Fast and Slow*.)

TypeSafe leaned into this and named their new type of classification models “System-One Models”, in contrast to text-generating Large Language Models (LLMs), which are more like System 2. Jev is their first System-One model release. (Almeida, “Introducing System-One Models and Jev”.)

How do they compare to LLMs?

LLMs work by predicting the most probable next token, step by step. Especially for machine-to-machine communication, people have tried to squeeze LLM output into a schema. There are techniques for that, but they limit the model in its reasoning capabilities (Almeida, “Introducing System-One Models and Jev”.) , and they are nowhere near efficient enough for realtime applications at a feasible cost.

So what does Jev do differently? It takes the same input as basic LLMs (which is plain text) and a set of questions with their answer candidates. Its output is then a set of probabilities for each question. The model fundamentally does not generate text and therefore cannot return an answer outside the supplied schema. (Which does **not** mean it cannot pick an incorrect answer.) That reliability and efficiency make it a good fit for realtime applications. (Almeida, “Introducing System-One Models and Jev”.)

	Large Language Model	System-One Model
Fundamental Output	Textstream ☰	Typed Probabilities 📊
Reasoning Efforts	High, Long Thinking 🧠	Low, Intuitive 💢
Cost Efficiency and Speed	Low, Slow 🐌	High, Fast 🚀

Comparison between System-One and LLMs (Almeida, “Introducing System-One Models and Jev”.)

Oh, and they are really really cheap. Jev is currently listed at 0.0047€ per 1M input tokens, with output tokens being free. TypeSafe attributes this price to the massive parallelism of its decision heads. (Almeida, “Introducing System-One Models and Jev”.)

Local Open-Weight alternatives

Despite Jev having been released only on September 15th, open-weight alternatives have sprouted up like mushrooms. They mostly work by architecturally modifying and fine-tuning a pre-trained model on a specific dataset. Since the problem is so much easier, way smaller models get acceptable results.

> **Open weights → Great news for privacy**
>
> TypeSafe AI claims not to train on your data (Almeida, “Introducing System-One Models and Jev”.) , which sounds good on paper. For highly sensitive applications, like handling health data, this is not enough. Open-weight models can be self-hosted, which gives the operator more control over where inference data goes. It does not guarantee that the whole application keeps no data off-premises.

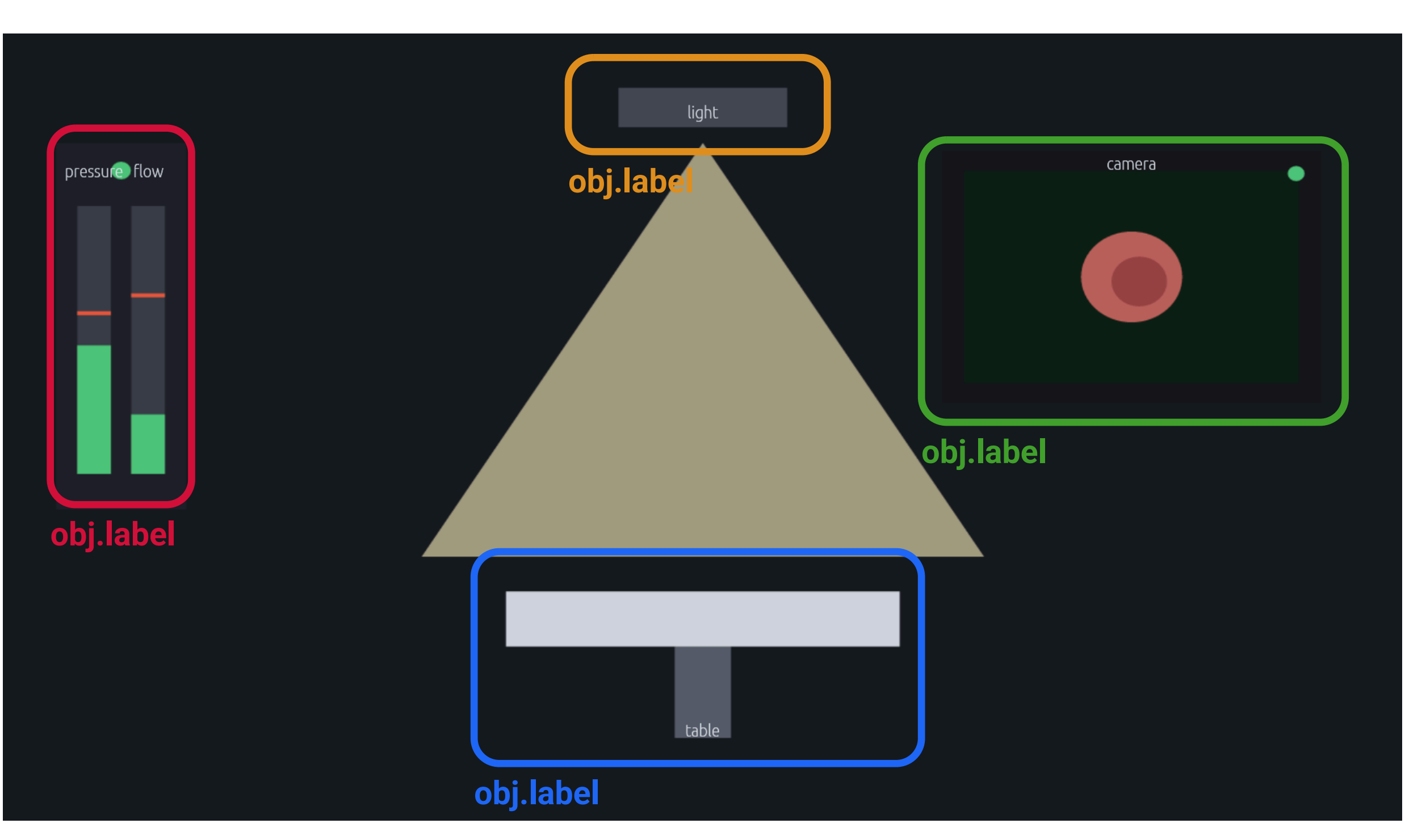
Some notable contenders are:
- JevK5 - 4B params, the open-weight benchmark leader when I started this experiment
- Kev - Similar design to JevK5
- DiffusionGemma - ~26B params, based on diffusion
- Laya - ~500MB size, less than 50ms latency even on CPU, often requires fine-tuning

Project introduction

The virtual operating room

Operating rooms have lots of equipment, and each device brings its own controls. A voice assistant could save you the constant reaching for knobs and menus. But a natural command is not enough on its own. The system has to identify which device and setting the speaker means, handle values precisely, and avoid changing anything the speaker did not ask to change.

For this experiment, I use a virtual room rather than actual equipment or a clinical workflow. It has an insufflator, an operating light, a surgical table, and an endoscope. Each object maps to a handful of state fields, such as gas flow, light brightness, table tilt, or endoscope zoom. Here is the room and the set of actions for each object.



The virtual operating room with the controlled equipment highlighted in color: insufflator, light, table, and endoscope.

Object	Example fields
obj.label	`gas_flow_l_min`, `target_pressure_mmhg`, `insufflator_active`
obj.label	`light_brightness`, `light_mode`
obj.label	`table_tilt_degrees`, `table_height_cm`
obj.label	`zoom_level`, `white_balance_locked`, `irrigation_active`

Equipment fields represented in the virtual room.

No patient, physical device, or clinical workflow is part of this experiment. I focus on the narrow question of whether a System-One model can turn short utterances into those room-state fields.

From utterance to room state

The API gives the model the current state, the utterance, and questions about possible field values. The choices are defined by the application. For example, a question could ask which of several light modes the speaker requested or what brightness value they want. Here is a request for “Zoom the camera in.” The response score of 3.79 is not a defined “one step in” command. Application logic still has to map it to a target and enforce allowed bounds. The example also asks about the light, where `null` means no requested change.

There are three kinds of questions in the API. The table below shows each one in context.

Primitive	Question	Example answer space	Example output
Choice	Which team should handle this ticket?	`billing`, `technical`, or `account`	`choice: "billing"`
Score	How frustrated is this customer?	0 = calm, 1 = frustrated, 2 = very frustrated	`score: 1.4`
Noul	Does this message request a refund?	True or false	`noul: 0.95`

System-One question types and example outputs.

```json
{
  "model": "jev-latest",
  "state": {
    "utterance": "Zoom the camera in",
    "camera_zoom": 3
  },
  "questions": {
    "camera_zoom": {
      "type": "score",
      "instructions": "Score the target zoom level of the camera",
      "criteria": [
        "The lowest zoom level",
        "Between lowest and medium zoom level",
        "Moderate zoom level",
        "Between medium and highest zoom level",
        "The highest zoom level"
      ]
    },
    "light_mode": {
      "type": "choice",
      "instructions": "Choose the target light mode based on the utterance",
      "criteria": {
        "normal": "Normal light operating mode",
        "cavity_focus": "Cavity focus light operating mode",
        "red_light": "Red light operating mode",
        "null": "No light change requested in the utterance"
      }
    },
    "light_activated": {
      "type": "noul",
      "instructions": "Should the light be activated or deactivated?"
    }
  }
}
```

```json
{
  "model": "jev-latest",
  "answers": {
    "camera_zoom": {
      "type": "score",
      "score": 3.79,
      "confidence": 0.95,
      "probabilities": {
        "0": 0.01,
        "1": 0.02,
        "2": 0.03,
        "3": 0.26,
        "4": 0.67,
        "5": 0.01
      }
    },
    "light_mode": {
      "type": "choice",
      "choice": "null",
      "confidence": 0.95,
      "probabilities": {
        "normal": 0.04,
        "cavity_focus": 0.01,
        "red_light": 0.01,
        "null": 0.94
      }
    },
    "light_activated": {
      "type": "noul",
      "noul": 0.921,
      "confidence": 0.921
    }
  }
}
```

A single utterance becomes typed field answers. The camera score is a prediction, not a command sent to equipment.

The model output is not the room state itself. Application code still has to interpret the answer, apply limits, and decide whether to act. That boundary matters: a valid `choice` cannot name an option outside its schema, but it can still choose the wrong option.

Defining the choices

For the benchmark, I describe possible values for each field and ask the model to select among them. Numeric commands need a little care. If someone says “turn the brightness up by twenty percent,” the model must infer the requested change, then choose a supported target. “Up” is relative to current state; `20` is not automatically the correct answer unless the application defines that mapping.

The criteria table shows candidate language for light brightness and table tilt. It shows what the classifier sees, not a complete specification of device safety limits.

	Base	20	95

light_brightness	`null`
	`increase:1`
	`decrease:1`
	`set:0`
	`set:100`
		`set:20`
		`increase:20`
		`decrease:20`
			`set:95`
			`increase:95`
			`decrease:95`

table_tilt	`null`
	`increase:1`
	`decrease:1`
	`set:0`
		`set:20`
		`increase:20`
		`decrease:20`
			`set:95`
			`increase:95`
			`decrease:95`

⋮	⋮	⋮	⋮

Illustrative candidate criteria for two numeric room fields.

Running locally

JevK5 includes a 2B-parameter model that runs on my RTX 2080. The server exposes a System-One-compatible API, so the surrounding request format stays close to the hosted Jev example. I use the model and package revision shown below; this is enough to reproduce the environment setup, but not the whole benchmark pipeline.

pyproject.toml

```toml
[project]
name = "jevk5-timerertim"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "jevk5[fast]",
    "causal_conv1d; sys_platform == 'linux'"
]

[build-system]
requires = ["uv_build>=0.12.17,<0.13.0"]
build-backend = "uv_build"

[tool.uv.sources]
causal-conv1d = { index = "astral-gpu-cu130" }
jevk5 = { git = "https://github.com/allebee/jevk5", rev = "v0.3.0" }

[[tool.uv.index]]
name = "astral-gpu-cu130"
url = "https://wheels.astral.sh/simple/cu130"
explicit = true
```

I start the server with `uv run jevk5-serve --model alibiserikbay/JevK5-2B --port 8090 --host 0.0.0.0`. The host option binds beyond localhost, so do not put this on an untrusted network without access controls.

What “local” does and does not mean

Running inference on hardware the hospital controls means the request does not need to be sent to a hosted model provider. That matters for sensitive health data. Local hosting is one important piece of a privacy-conscious system, alongside the hospital’s existing rules for data access and handling.

Evaluation

Dataset and procedure

I use five virtual surgery scenarios: appendectomy, hernia repair, fundoplication, cholecystectomy, and sleeve gastrectomy. The benchmark setup uses 1,000 generated cases with 12 utterances each, including small talk that should not change room state. The benchmark uses `inclusionai/ling-3.0-flash-vl` as the generator for cases and expected output states. That makes a large test set quickly, but generated labels are not the same thing as expert-reviewed clinical ground truth.

I evaluate each utterance on its own and count the predicted field values that match the expected values. The comparison data contains 12 000 Jev entries and 2256 JevK5-2B entries from 188 local-model cases. The paired comparison uses those same entries. JevK5-2B is correct where Jev is not on 4 entries. Jev is correct where JevK5-2B is not on 710.

Overall results

	Jev (TypeSafe AI)	JevK5-2B 🖥
Field Macro-F1	**0.886**	0.574
Fresh Accuracy	**0.887**	0.573
Hallucination Rate	**0.680** %	8.47 %
Fresh Latency p50 / p95	**260 ms** / **358 ms**	15 100 ms / 19 300 ms
Fresh Calibration ECE	**0.0351**	0.287

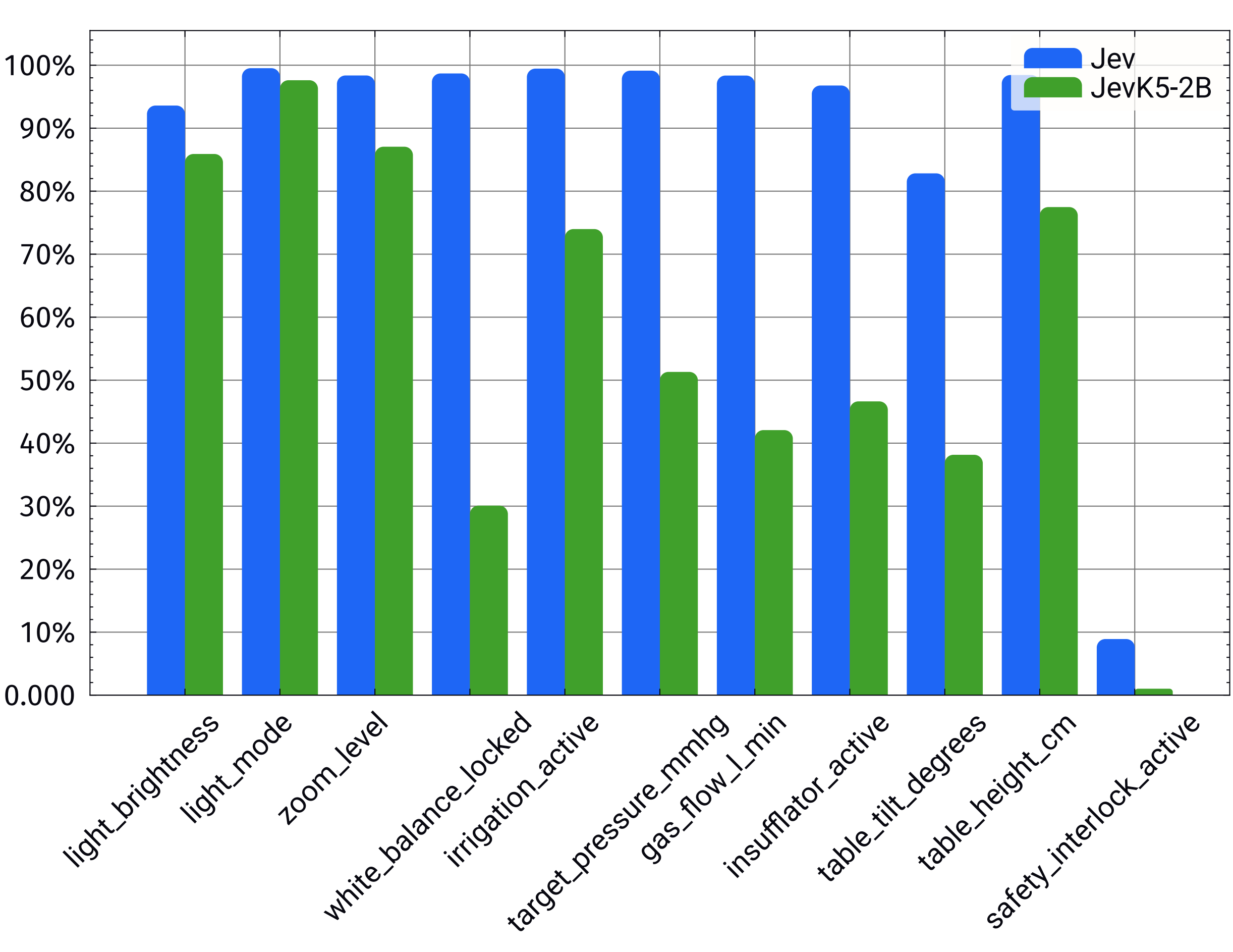
Recorded utterance-level accuracy and other metrics for Jev and JevK5-2B.

In the recorded runs, Jev performs better than JevK5-2B on utterance-level accuracy. Jev also performs better in the rolling evaluation, where predictions are applied through an utterance sequence and earlier mistakes can affect later state. I keep these measurements separate because JevK5-2B has fewer evaluated cases. The paired comparison uses the same cases for both models and points in the same direction.

For the full comparison, the field accuracy is 88.7 % for Jev and 57.3 % for JevK5-2B. In rolling mode, it is 61.9 % and 8.78 %, respectively. The paired comparison above says the same.

The table’s field macro-F1 averages per-field F1 scores, so a field with many examples does not drown out one with poor performance. The models do not have the same coverage in every sample set, so treat this as a summary of the supplied measurements, not a controlled clinical comparison.

It depends on the field



Field-level F1 scores from the recorded comparison data. Each bar measures classification quality for one state field.

Jev handles several fields well in this generated set, but safety interlock activation is a glaring exception: its F1 is 8.91 %, with recall of 4.66 %. JevK5-2B does even worse on that field, with F1 of 1.02 % and recall of 0.513 %. Both models miss nearly all positive interlock cases. For a control system, that alone kills any idea of letting a model own safety decisions. Table tilt is another weaker field for Jev, at F1 82.8 %.

There may be a dataset issue here too. For example, an utterance like “tilt table by 5 degrees” could have an expected state of `table_tilt: -5`, depending on how direction was interpreted. If expected signs are inconsistent with the utterance, that would lower the score without being a model error. I have not verified this explains the table-tilt result, so it needs checking against the cases before drawing conclusions.

Aggregate scores can hide exactly these failures.

Fast enough, on which machine?

Jev’s recorded fresh p50/p95 latency is 261 ms / 359 ms per utterance. JevK5-2B’s is 15 200 ms / 19 600 ms. So the local model ran on my RTX 2080, but this particular 2B configuration was not fast enough for the near-real-time interaction I had in mind. “It runs locally” and “it is useful locally” are different claims.

Conclusion

System-One’s constrained outputs are an appealing interface for equipment-control software: the schema narrows what the model can return, and the supplied API can score multiple fields directly. My experiment does not show that JevK5-2B is a practical local substitute for Jev. It shows the opposite for this task and setup: lower accuracy, a very weak safety-interlock field, and much higher latency.

This is a virtual benchmark built from generated cases. It says nothing about safety, clinical usefulness, or performance with real operating-room speech. Before a model could touch real equipment, a production design would need independent safety controls, proper handling of uncertainty, and validation with domain experts.

The next step is not to let an LLM near the operating-room switches. It is to find out whether better data, tighter task definitions, and a faster local model can close the gap without weakening the safety boundary.

References

Almeida, Diogo. “Introducing System-One Models and Jev.” Accessed October 10, 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev.

Kahneman, Daniel. *Thinking, Fast and Slow*. First paperback. Farrar, Straus, Giroux, 2011.

https://github.com/allebee/jevk5.

https://github.com/jaredpalmer/kev.

https://huggingface.co/google/diffusiongemma-26B-A4B-it.

https://github.com/NandhaKishorM/laya.